TASK - 12 - PUBLIC WEB SERVER AND PRIVATE DATABASE SERVER ON AWS

OBJECTIVE

Create two EC2 instances, one in a public subnet and one in a private subnet.

Host a website on the public Web Server.

Host a database on the private Database Server.

Allow the Database Server to be accessed only from the Web Server and prevent public access to the Database Server, even if the Security Group contains open entries.

Provide internet access to the private Database Server through the NAT Gateway so that software packages can be updated and patched.

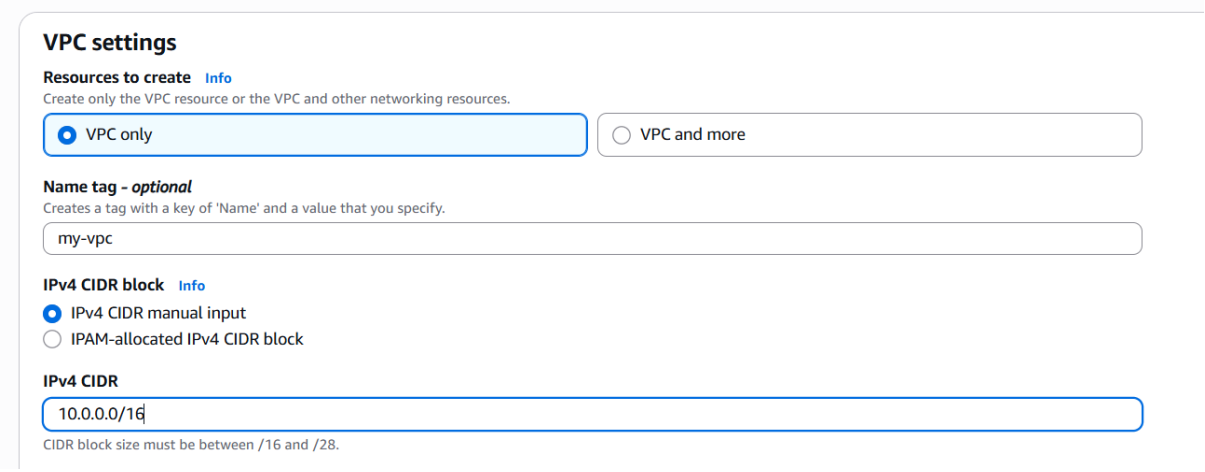
PART 1 — VPC AND SUBNET CONFIGURATION

Step 1 — Create a VPC

A VPC named `my-vpc` was created with the CIDR block:

```
10.0.0.0/16
```

This VPC provides the private network for the Web Server and Database Server.



Step 2 — Create the Public Subnet

A public subnet named `public` was created with:

`10.0.1.0/24`

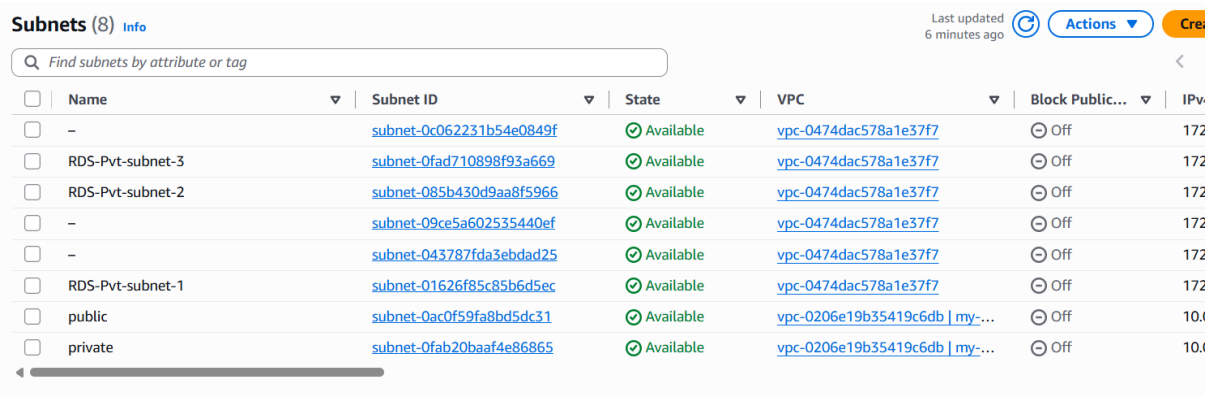
The public subnet is used for the Web Server.

Step 3 — Create the Private Subnet

A private subnet named `private` was created with:

`10.0.2.0/24`

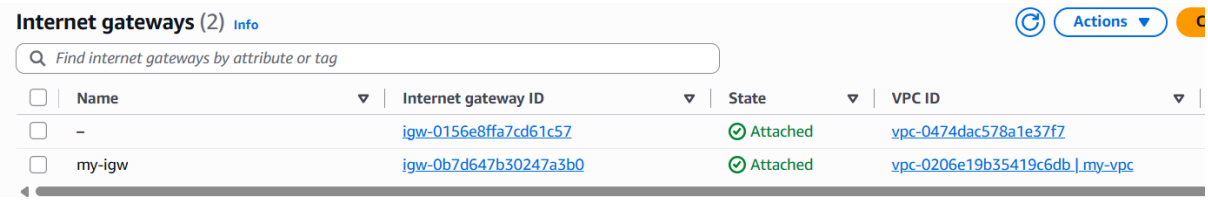
The private subnet is used for the Database Server.



Step 4 — Create Internet Gateway

An Internet Gateway named `my-igw` was created and attached to the VPC `my-vpc`.

The Internet Gateway provides internet connectivity for resources in the public subnet through the public route table.



Step 5 — Configure Public Route Table

A route table named `my-public-rt` was created for the public subnet.

The public subnet was associated with this route table.

The following default route was configured:

```
Destination: 0.0.0.0/0
Target: my-igw
```

This allows the Web Server in the public subnet to communicate with the internet.

The traffic flow is:

```
Internet
    ↓
Internet Gateway
    ↓
my-public-rt
    ↓
Public Subnet
    ↓
Web Server
```

Step 6 — Configure Private Route Table

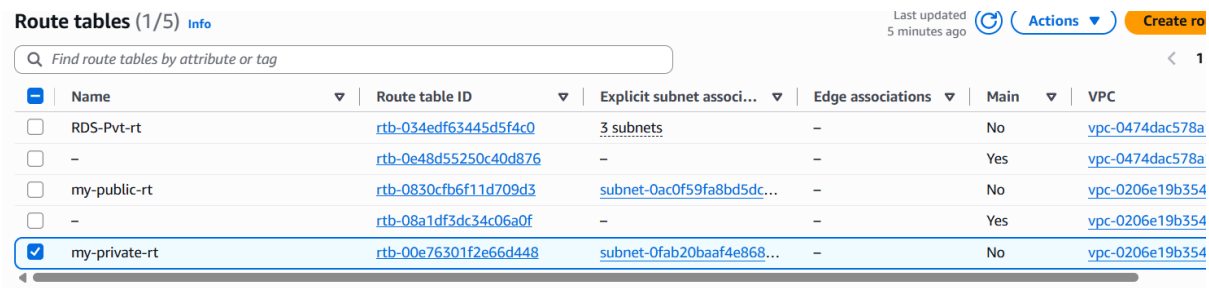
A route table named `my-private-rt` was created for the private subnet.

The private subnet was associated with this route table.

The default route was configured to use the NAT Gateway:

```
Destination: 0.0.0.0/0
Target: NAT Gateway
```

This allows the Database Server to access the internet for outbound connections without requiring a public IP address.



PART 3 — NETWORK ACL CONFIGURATION

Step 7 — Configure Public NACL

A Network ACL named `mynacl-public` was created and associated with the public subnet.

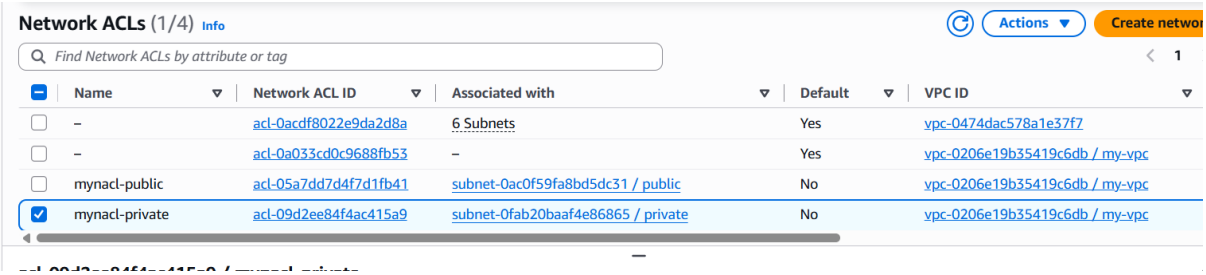
The required inbound and outbound traffic was allowed in the NACL for the practical.

Step 8 — Configure Private NACL

A Network ACL named `mynacl-private` was created and associated with the private subnet.

The required inbound and outbound traffic was allowed in the NACL for the practical.

The NACL configuration was used to control traffic entering and leaving the private subnet.



PART 4 — NAT GATEWAY CONFIGURATION

Step 9 — Create NAT Gateway

A NAT Gateway was created in the public subnet.

The NAT Gateway was used to provide outbound internet access to the Database Server in the private subnet.

The Database Server does not require a public IP address to access the internet through the NAT Gateway.

The traffic flow is:

```
Database Server
10.0.2.144
      ↓
NAT Gateway
```

```
      ↓
Internet Gateway
      ↓
Internet
```

PART 5 — EC2 INSTANCE CONFIGURATION

Step 10 — Create Web Server

An EC2 instance was launched in the public subnet.

The Web Server received the private IP address:

```
10.0.1.37
```

The Web Server was configured with public access so that the WordPress website could be accessed from the internet.

Step 11 — Create Database Server

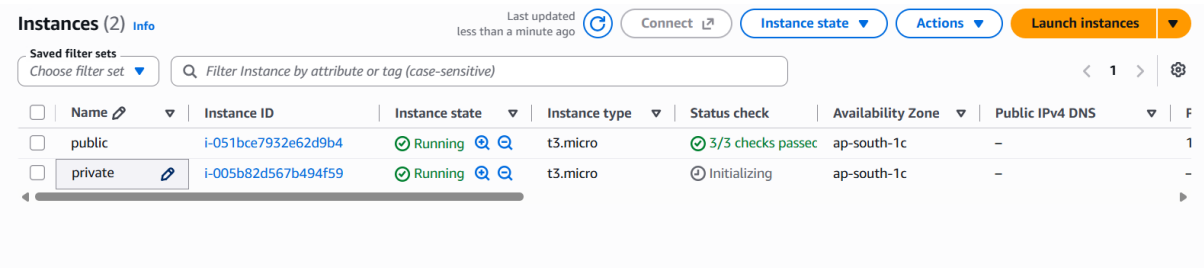
A second EC2 instance was launched in the private subnet.

The Database Server received the private IP address:

```
10.0.2.144
```

The Database Server was kept in the private subnet without a public IP address.

Therefore, the Database Server was not directly exposed to the public internet.



PART 6 — SECURITY GROUP CONFIGURATION

Step 12 — Configure Web Server Security Group

The Web Server Security Group was configured to allow HTTP traffic:

```
Type: HTTP
Protocol: TCP
Port: 80
```

```
Source: 0.0.0.0/0
```

SSH access was configured for server administration:

```
Type: SSH
Protocol: TCP
Port: 22
Source: My IP
```

Step 13 — Configure Database Server Security Group

The Database Server Security Group was configured to allow MySQL/MariaDB traffic only from the Web Server Security Group.

```
Type: MySQL/Aurora
Protocol: TCP
Port: 3306
Source: Web Server Security Group
```

This allows the Web Server to communicate with the Database Server while preventing direct database access from the public internet.

PART 7 — CONNECT TO THE PRIVATE DATABASE SERVER

Step 14 — Create **`test.pem`** from the Existing Key

The existing private key `key-1.pem` was opened in Notepad and the complete key contents were copied.

A new file named `test.pem` was created from the terminal:

```
nano test.pem
```

The copied contents of `key-1.pem` were pasted into `test.pem`.

The file was saved and exited.

The required permission was assigned:

```
chmod 400 test.pem
```

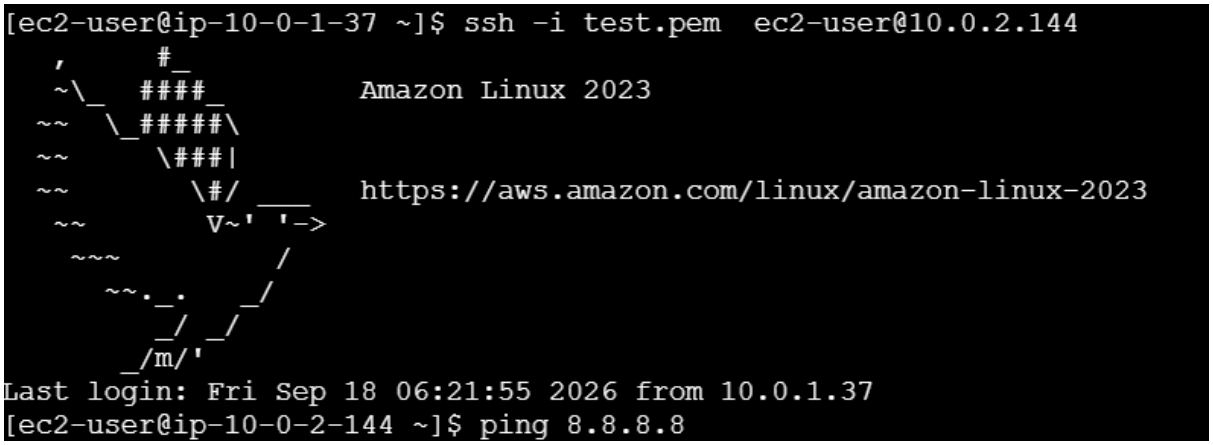
Step 15 — Connect to the Private Database Server Using SSH

From the public Web Server, SSH access was established to the private Database Server using its private IP address:

```
ssh -i test.pem ec2-user@10.0.2.144
```

The connection was successful.

This demonstrates that the private Database Server can be accessed from the Web Server through the private network.



PART 8 — WEB SERVER CONFIGURATION

Step 16 — Update the Web Server

The Web Server software packages were updated:

```
sudo dnf update -y
```

Step 17 — Install and Start Apache

Apache HTTP Server was installed:

```
sudo dnf install -y httpd
```

Apache was enabled and started:

```
sudo systemctl enable --now httpd
```

Step 18 — Install PHP

The required PHP packages for WordPress were installed:

```
sudo dnf install -y php php-mysqlnd php-gd php-mbstring php-xml php-json php-curl
```

Apache was restarted after installing PHP:

```
sudo systemctl restart httpd
```

Step 19 — Download and Install WordPress

The WordPress package was downloaded:

```
cd /tmp
wget https://wordpress.org/latest.tar.gz
```

The downloaded archive was extracted:

```
tar -xzf latest.tar.gz
```

The WordPress files were copied to the Apache document root:

```
sudo cp -r /tmp/wordpress/* /var/www/html/
```

Step 20 — Configure WordPress File Permissions

Ownership of the WordPress files was changed to the Apache user:

```
sudo chown -R apache:apache /var/www/html/
```

The required permissions were assigned:

```
sudo chmod -R 755 /var/www/html/
```

Apache was restarted:

```
sudo systemctl restart httpd
```

PART 9 — PRIVATE DATABASE SERVER CONFIGURATION

Step 21 — Update the Database Server

After connecting to the private Database Server, the software packages were updated:

```
sudo dnf update -y
```

This verifies that the private server can access the internet through the NAT Gateway.

Step 22 — Install MariaDB

MariaDB Server was installed:

```
sudo dnf install -y mariadb105-server
```

The MariaDB service was enabled and started:

```
sudo systemctl enable --now mariadb
```

Step 23 — Create the WordPress Database

MariaDB was opened:

```
sudo mysql
```

A database named `wordpress` was created:

```
CREATE DATABASE wordpress;
```

A database user was created for the Web Server:

```
CREATE USER 'wpuser'@'10.0.1.37' IDENTIFIED BY 'Wp@ss12345';
```

The required privileges were granted:

```
GRANT ALL PRIVILEGES ON wordpress.* TO 'wpuser'@'10.0.1.37';
```

The privileges were applied:

```
FLUSH PRIVILEGES;
```

The MariaDB session was then exited.

The database user was restricted to the Web Server private IP address `10.0.1.37`.

PART 10 — CONNECT THE WEB SERVER TO THE DATABASE SERVER

Step 24 — Install the MariaDB Client

The MariaDB client was installed on the Web Server:

```
sudo dnf install -y mariadb105
```

Step 25 — Connect to the Private Database

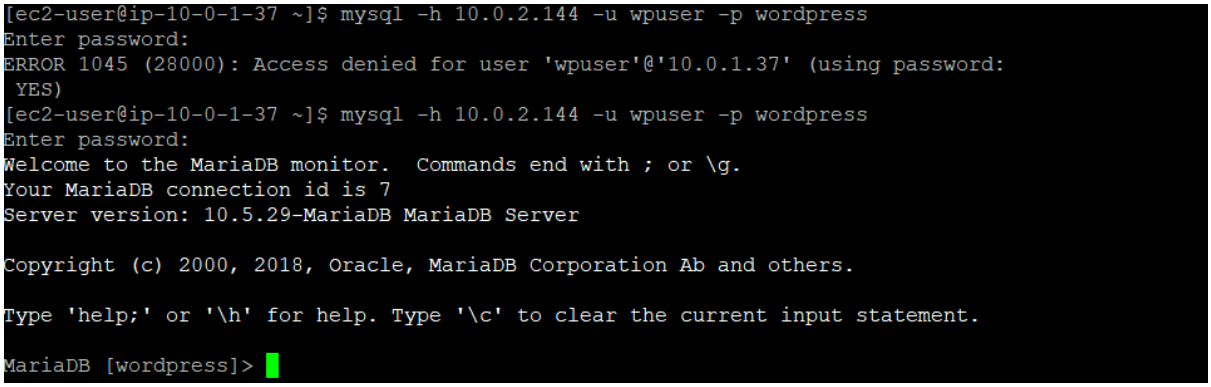
The Web Server was connected to the MariaDB database using the Database Server's private IP address:

```
mysql -h 10.0.2.144 -u wpuser -p wordpress
```

The database password was entered when prompted.

The connection was successful.

This verified communication between the Web Server and the private Database Server through the private network.



Step 26 — Configure WordPress Database Connection

The WordPress configuration file was created:

```
cd /var/www/html
sudo cp wp-config-sample.php wp-config.php
```

The configuration file was edited:

```
sudo vim wp-config.php
```

The database details were configured as follows:

```
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'wpuser' );
define( 'DB_PASSWORD', 'Wp@ss12345' );
define( 'DB_HOST', '10.0.2.144' );
```

The WordPress files were assigned the required ownership and permissions:

```
sudo chown -R apache:apache /var/www/html/
sudo chmod -R 755 /var/www/html/
```

Apache was restarted:

```
sudo systemctl restart httpd
```

PART 11 — WORDPRESS VERIFICATION

Step 27 — Verify the WordPress Website

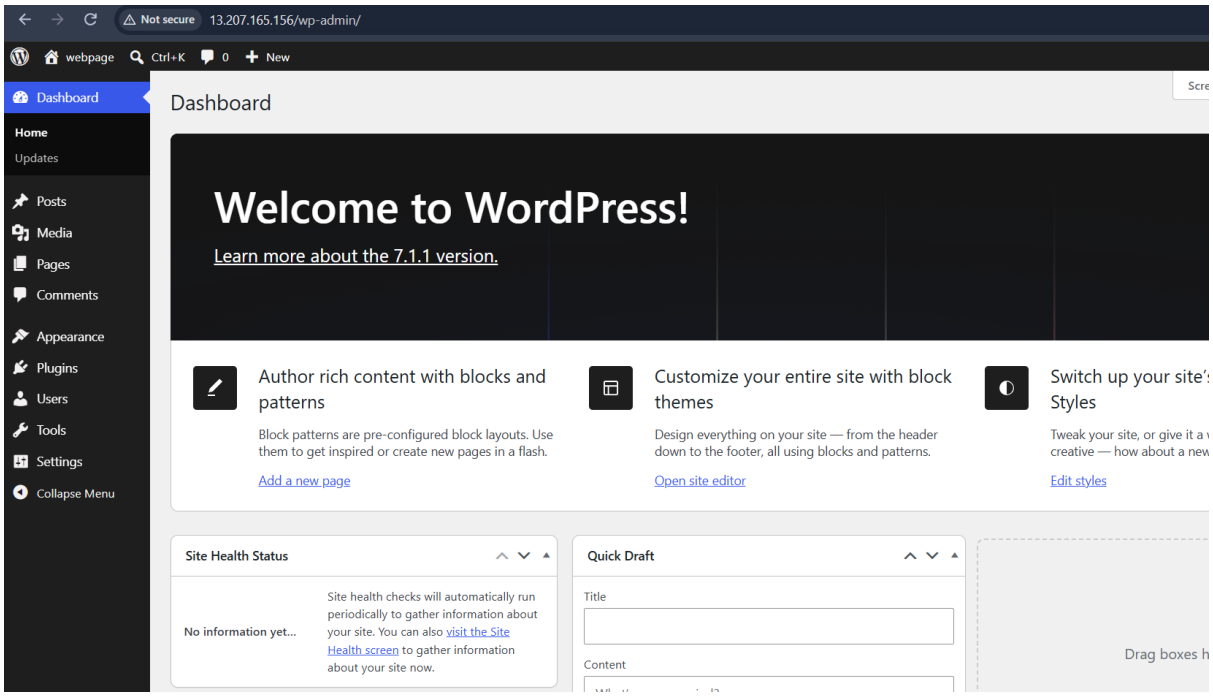
The Web Server's public IP address was opened in a web browser.

The WordPress setup page was displayed and the WordPress installation was completed.

The WordPress Dashboard was successfully accessed.

This confirmed that:

- WordPress was hosted on the public Web Server.
- The Web Server could communicate with the private Database Server.
- MariaDB was running on the private Database Server.
- WordPress was successfully connected to the private database.



CONCLUSION

The AWS environment was successfully configured with a public Web Server and a private Database Server. WordPress was hosted on the public Web Server, while MariaDB was configured on the private Database Server. The Web Server communicated with the Database Server through private networking, and database access was restricted to the Web Server. The private Database Server was provided outbound internet access through a NAT Gateway, allowing software updates and patching without exposing the Database Server directly to the public internet.